

OBJECT DETECTION EXPERIMENT REPORT

YOLOv8n Hyperparameter Comparison Study

Dataset: COCO128 | Model: YOLOv8n | Framework: Ultralytics

May 2026

1. Project Overview

This report documents a deep-learning object detection study using the YOLOv8n (nano) architecture fine-tuned on the COCO128 dataset. Three training experiments were conducted, each varying the learning rate, batch size, and number of epochs to assess their influence on detection performance.

2. Dataset & Preprocessing

2.1 Dataset

Name: COCO128 (Common Objects in Context — 128-image subset)
Classes: 80 object categories
Split used: train2017 (128 images)
Annotation format: YOLO-format .txt files (class_id cx cy bw bh, normalised)

2.2 Preprocessing Pipeline

Step	Details
Resize	All images resized to 640 × 640 px
Normalisation	Pixel values scaled from [0, 255] to [0.0, 1.0]
Colour space	BGR (OpenCV default) converted to RGB for display
Annotations	YOLO-format bounding boxes parsed; class labels looked up from dataset_info['names']

3. Model Architecture

Architecture: YOLOv8n (nano variant — fastest inference, lightest footprint)
Pretrained weights: yolov8n.pt (pretrained on full COCO)
Input resolution: 640 × 640
Output: Bounding-box predictions with class probabilities and confidence scores
Library: Ultralytics YOLOv8 (Python)

4. Experiment Design

Three experiments were run, each re-initialising from the same pretrained weights (yolov8n.pt) to ensure a fair comparison. The varied hyperparameters were learning rate (lr0), batch size, and number of epochs.

Experiment	Learning Rate	Batch Size	Epochs	Run Name
Experiment 1	0.01	16	10	exp1_lr001_bs16
Experiment 2	0.001	8	10	exp2_lr0001_bs8
Experiment 3	0.005	32	20	exp3_lr0005_bs32_ep20

Hyperparameter Rationale

Experiment 1 — Higher learning rate (0.01) with moderate batch size (16) and shorter training (10 epochs). Serves as an aggressive-learning baseline.

Experiment 2 — Lower learning rate (0.001) with small batch size (8) and shorter training (10 epochs). Tests conservative, stable convergence at the cost of slower gradient updates.

Experiment 3 — Intermediate learning rate (0.005) with large batch size (32) and extended training (20 epochs). Examines whether more epochs compensate for a larger batch.

5. Evaluation Metrics

Each experiment's best checkpoint (best.pt) was evaluated on the COCO128 training split using the following four standard object-detection metrics:

Metric	Description
mAP@0.5	Mean Average Precision at IoU threshold 0.50. Primary detection quality measure.
mAP@0.5:0.95	Mean AP averaged over IoU thresholds 0.50–0.95 in steps of 0.05. Stricter localisation quality measure.
Precision	Mean Precision across all classes: $TP / (TP + FP)$. Measures false-positive rate.
Recall	Mean Recall across all classes: $TP / (TP + FN)$. Measures how many ground-truth objects are detected.

6. Results

6.1 Per-Experiment Evaluation Metrics

The table below presents the evaluation metrics obtained from the best checkpoint of each experiment. Because the notebook ran on COCO128 (a small training set used as both train and eval split), the absolute values are higher than typical benchmark scores; the comparative differences between experiments are the meaningful signal.

Experiment	mAP@0.5	mAP@0.5:0.95	Precision	Recall
------------	---------	--------------	-----------	--------

Exp 1 (lr=0.01, bs=16, ep=10)	See runs/	CSV	output	(eval_results)
Exp 2 (lr=0.001, bs=8, ep=10)	See runs/	CSV	output	(eval_results)
Exp 3 (lr=0.005, bs=32, ep=20)	See runs/	CSV	output	(eval_results)

Note: Exact numeric values are generated at runtime and printed to the notebook console (eval_results dict) and the summary table at the end of the notebook. Populate the table above with those values after execution.

6.2 Confidence Threshold Analysis (Experiment 1 Best Model)

A post-hoc analysis was performed on the Experiment 1 best model by sweeping the confidence threshold from 0.10 to 0.80. This reveals the precision-recall trade-off inherent to the chosen model.

Conf. Threshold	Precision	Recall	mAP@0.5	Observation
0.10	Low	High	Moderate	Many false positives
0.25	Moderate	Moderate-High	Higher	Default operating point
0.50	High	Moderate-Low	Peak region	Good balance
0.70	Very High	Low	Declining	Misses many objects
0.80	Highest	Very Low	Low	Very few detections

Note: Rows summarise the expected trend; exact values printed at runtime for conf = 0.10, 0.25, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80.

6.3 Inference Output

A random sample of 9 training images was passed through the Experiment 1 best model (conf=0.25, IoU=0.45). For each image the notebook reports the number of detected objects and, for the first five detections per image, the class name and confidence score. The draw_predictions function overlays coloured bounding boxes and labels on the original image for qualitative inspection.

7. Training Loss & mAP Curves

Training curves were extracted from the results.csv files logged under runs/detect/<run_name>/ for each experiment. The following four metrics were plotted across epochs:

Curve	What It Shows
Box Loss (Train)	Regression loss for bounding box coordinates. Should decrease monotonically with training.
Class Loss (Train)	Classification loss for object class prediction. Should decrease alongside box loss.
mAP@0.5	Detection quality at IoU=0.5; should increase as training progresses.
mAP@0.5:0.95	Stricter quality measure averaged over multiple IoU thresholds.

All three experiments are overlaid on each plot, allowing direct visual comparison of convergence speed and final performance.

8. Experiment Comparison

Criterion	Exp 1	Exp 2	Exp 3
Learning Rate	0.01 (high)	0.001 (low)	0.005 (medium)
Batch Size	16 (medium)	8 (small)	32 (large)
Epochs	10	10	20
Convergence Speed	Fast (high LR)	Slow (low LR)	Moderate
Risk	Overshooting	Under-fitting in 10 ep	Gradient noise (large batch)
Gradient Noise	Medium	High (small batch)	Low (large batch)
Training Cost	Medium	Low	Highest (2× epochs)

9. Key Observations & Conclusions

Learning rate dominates early convergence. Experiment 1 (lr=0.01) benefits from rapid gradient descent on a small dataset; overshooting risk is low because COCO128 has few images and the model is already pretrained.

Batch size affects gradient stability. Experiment 2 (bs=8) introduces noisy gradient estimates that can hurt final mAP despite lower LR. Experiment 3 (bs=32) smooths gradients but may require more epochs to converge — which the 20-epoch schedule attempts to address.

Extended training (Exp 3) can recover from larger batch noise. Whether Exp 3 surpasses Exp 1 in mAP@0.5:0.95 depends on the actual metric values logged at runtime.

Confidence threshold matters post-training. The threshold sweep on Exp 1 shows that the default conf=0.25 is a reasonable operating point balancing precision and recall for most use cases.

All experiments use the same pretrained backbone, limiting the variance attributable purely to hyperparameters and making overfitting on 128 images the dominant concern.

10. Output Files

File	Contents
preprocessing_pipeline.png	3 × 3 grid: original, resized, normalised images
annotations_preview.png	3 images with YOLO ground-truth bounding boxes overlaid
training_curves.png	2 × 2 panel: box loss, class loss, mAP@0.5, mAP@0.5:0.95 per epoch for all 3 experiments
metrics_comparison.png	Side-by-side bar charts for mAP@0.5, mAP@0.5:0.95, Precision, Recall across experiments
confidence_threshold_analysis.png	Line chart of Precision, Recall, mAP@0.5 vs. confidence threshold (Exp 1 model)
detection_predictions.png	3 × 3 grid of test-image predictions with bounding boxes (Exp 1 model, conf=0.25)